

Storage Connector – Project Report

Alexandro Bolfa 331500

Steen Krøyer - STEK

23698 characters

Software Technology Engineering

Semester 7

01.12.2025

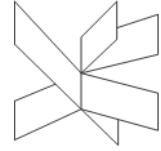
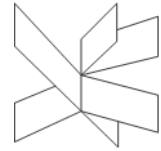
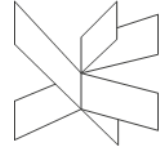


Table of content

1. Abstract	1
2. Introduction	2
3. Analysis	3
3.1 Requirements	3
3.1.1 Functional requirements	3
3.1.2 Non-functional requirements	4
3.2 Use case	4
3.3 Use case description	7
3.4 System sequence diagram	8
3.5 Domain model	9
3.6 Entity Relationship Diagram	10
3.7 Security	11
4. Design	12
4.1 Architectural overview	12
4.2 Client	12
4.3 Gateway	13
4.4 Services	13
4.5 Persistence	14
4.6 Security	14
5. Implementation	14
5.1 Gateway edge (routing, authentication, resilience)	14
5.2 LinkingService API	17

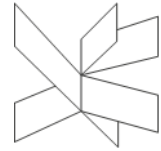


5.3	LinkingService core	17
5.4	OAuth clients and refresh	20
5.5	File aggregation.....	22
5.6	Identity touchpoints	26
5.7	Resilience and security.....	26
6.	Testing.....	26
6.1	Acceptance testing	26
6.2	Backend testing.....	27
6.3	Frontend testing	27
6.4	Summary of test coverage.....	28
7.	Discussion and conclusion.....	28
8.	References	29
	List of Figures	30
	List of Tables	30
	Appendices.....	31



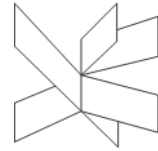
1. Abstract

Storage Connector is a secure system which works as a microservice-based web app. It integrates Google Drive and OneDrive into a unified file browser. Users can view files but not edit them. Its architecture has two deployable .NET9 services: IdentityService and LinkingService. It uses a YARP-based API Gateway with resilience policies and a React frontend using TanStack Query. The platform includes user registration and email verification and authentication. It also manages passwords and links accounts for Google and Microsoft through OAuth2.0 with PKCE. Users can check connection status and browse or open files from both providers. The system has security features which include JWT authentication and token introspection. It also encrypts OAuth tokens when stored and uses IP rate limiting and CORS restrictions. Security headers are used as well.



2. Introduction

Nowadays, people and companies use various cloud storage solutions like Google Drive and OneDrive and SharePoint to store and manage files. These services offer secure data and less delay and cost efficiency and access from anywhere with an internet connection (Barry, P.). However, managing data across different platforms can be challenging because of inconsistent interfaces and finding files quickly can be inefficient (Gartner). So, users often must switch between several services which causes a drop in productivity. This project plans to fix the issue of separate cloud storages by creating a web application which merges and organizes files from different cloud services into one interface. Users will be able to connect their cloud accounts and see all stored files in a single dashboard which makes access easier and boosts workflow efficiency.



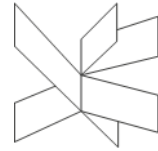
3. Analysis

3.1 Requirements

Based on the project's goal, 13 functional requirements have been defined and implemented.

3.1.1 Functional requirements

1. As a user, I want to create an account with email and password so I can use the app.
2. As a user, I want to verify my email, so my account is activated.
3. As a user, I want to log in with my email and password so I can access my account.
4. As a user, I want to reset my password so that I can regain access if I forget it.
5. As a user I want to log out, so that this device no longer has access.
6. As a user, I want to link my provider account (Google/Microsoft) so the app can read my Drive files.
7. As a user, I want to see each provider's connection state so that I know what is available.
8. As a user I want expired connections to recover so that I can keep using the app.
9. As a user, I want to disconnect a provider so the app stops accessing it.
10. As a user, I want one dashboard listing files from all connected providers so that I don't switch apps.
11. As a user, I want to search by file name across providers so that I can find files quickly.
12. As a user, I want to see the file's metadata (size, last modified date, file type), so that I can quickly verify it is the correct item before opening it.
13. As a user, I want to open a file in its native web app in a new tab so I can view it there.



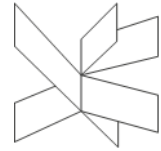
3.1.2 Non-functional requirements

1. The system must be split into independently deployable services with isolated databases.
2. The system must include an authentication mechanism for users and services.
3. The system must include authenticated server-to-server communication between services.
4. The system must rely on a defined protocol for all inter-service and client-service communication.
5. The system must include a GUI for client-facing interactions.

3.2 Use case

The previously listed functional requirements have been merged into eleven use cases:

- Register an account
- Verify email
- Log in
- Reset password
- Log out
- Link provider's account
- Manage provider's connection
- Browse unified files
- Cross-provider search
- See item's metadata
- Open item in provider



The use case diagram shows how the user interacts with the system's use cases. There is just one actor which is the user. Other actors such as an administrator, are irrelevant, since the system focuses on making user tasks easier and looks only at interactions between the user and their files.

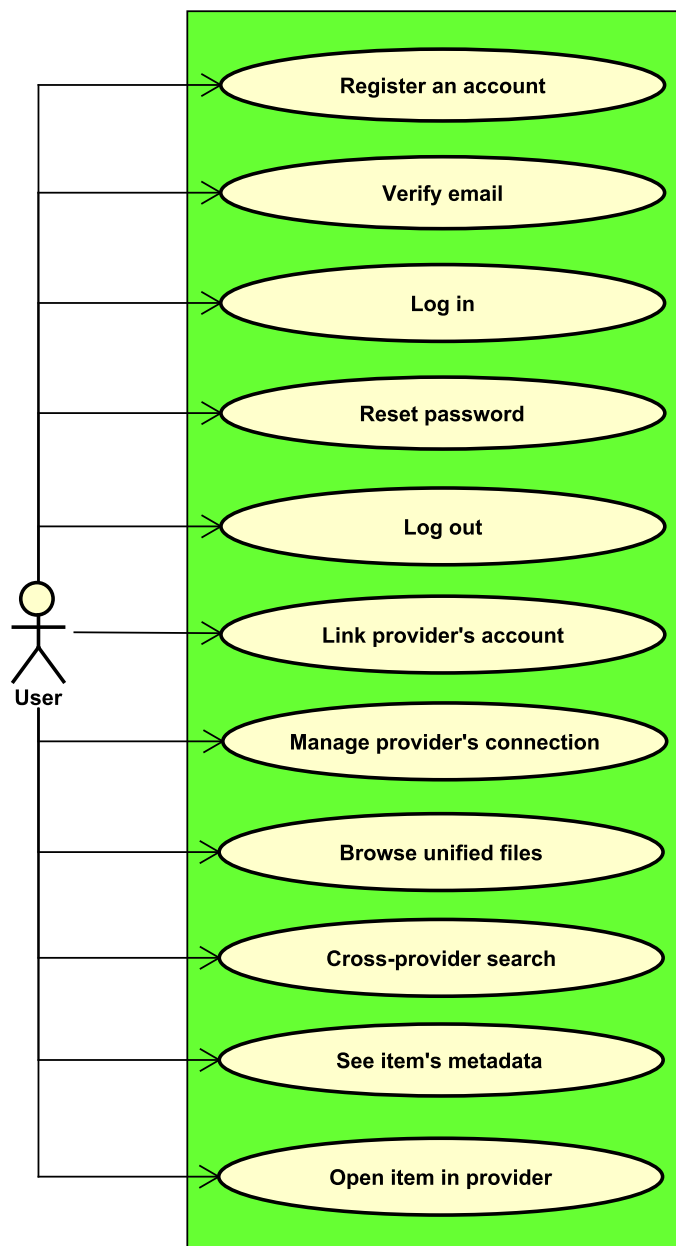
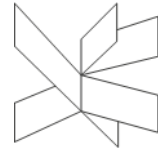
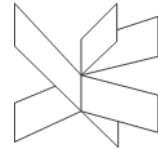


Figure 1: Use case diagram



Use Case:	Covered requirements:
Register an account	1
Verify email	2
Log in	3
Reset password	4
Log out	5
Link provider's account	6
Manage provider connection	7, 8, 9
Browse unified files	10
Cross-provider search	11
See item's metadata	12
Open item in provider	13

Table 1: Requirements coverage

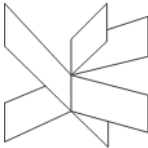


3.3 Use case description

The "Link provider's account" use case shows what steps a user needs to take to connect a provider account to their system account. The remaining use case descriptions are attached in the appendix.

Use case 6	Link provider's account
Summary	Link Google Drive/ OneDrive for read-only access.
Actor	User
Precondition	User is logged in.
Postcondition	Google/Microsoft tokens stored securely; status Connected.
Base sequence	1. User selects provider. 2. User connects to chosen provider. 3. System redirects to provider's consent for read only access [ALT1]. 4. User consents; provider redirects back with code [ALT2]. 5. System exchanges code for tokens; stores encrypted [ALT3]. 6. System confirms connection with provider.
Alternate sequence (branch or exception)	[ALT1] User denies consent (go back to 1). [ALT2] Code missing/invalid (go back to 1). [ALT3] Token exchange fails (go back to 1). [*ALT0] Can cancel at any time.
Note	Covers requirement 6

Table 2: Use case description - Link provider's account



3.4 System sequence diagram

The diagram illustrates the sequence of user interactions with the system. First, users request to link a provider account. The system then sends users to the provider's consent page. When access is approved by the provider, it sends users back to the system along with the approval result. The system then saves the connection for the user's account by storing the necessary tokens and permissions. Lastly, the system confirms the connection was successful.

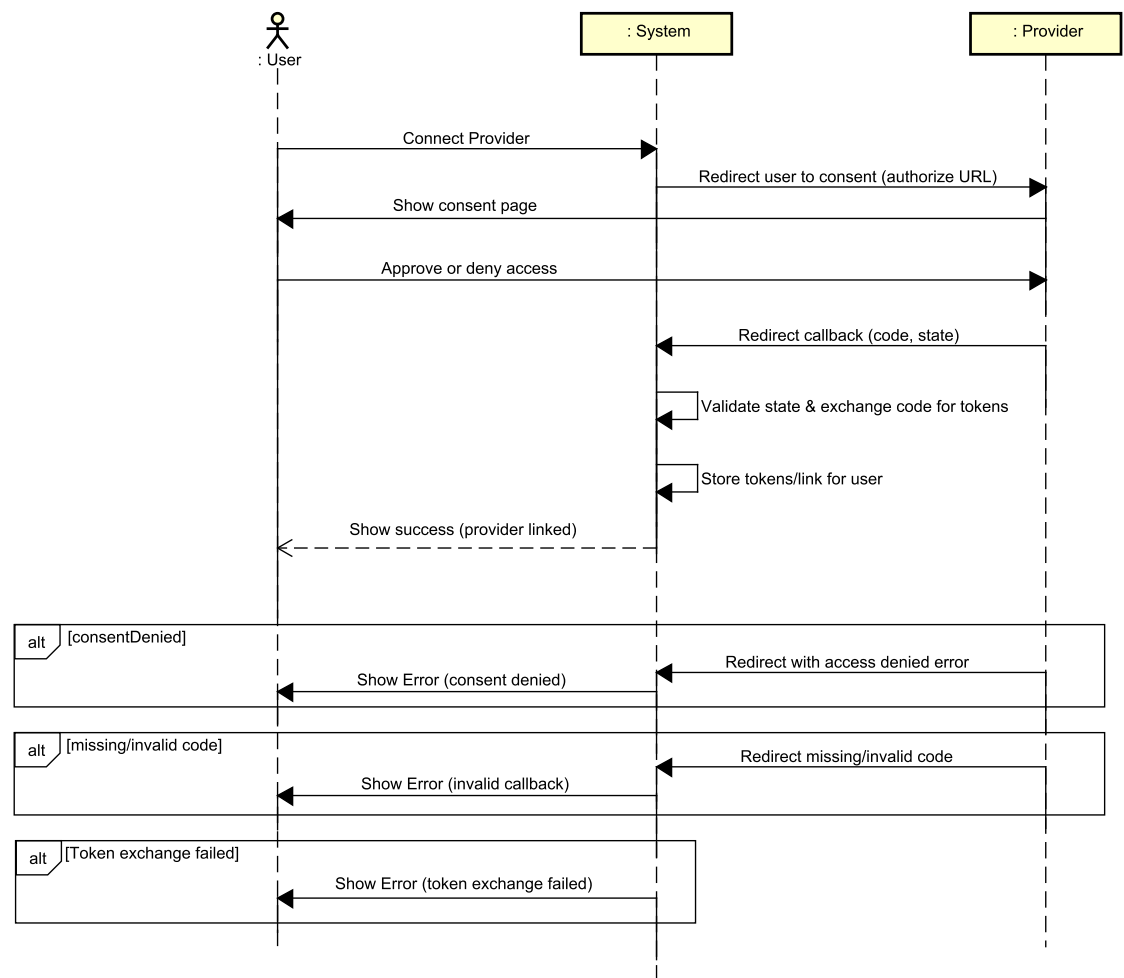
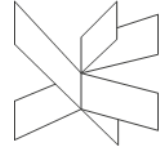


Figure 2: System sequence diagram - Link provider's account



3.5 Domain model

The domain model is built around the User. Users have email confirmation and password reset tokens that expire after a limited time and get flagged when used. Users can connect one account for each provider, such as Google and Microsoft. Each linked account keeps the provider type, the encrypted refresh token, its expiration time and the related scopes. The *TokenSet* given by the provider, which includes the access token, refresh token, expiry and scopes, is temporary and not stored directly. Only the refresh token, expiry and scopes are stored in the linked account. The *ProviderType* is an enumeration showing if the link is for Google or Microsoft account.

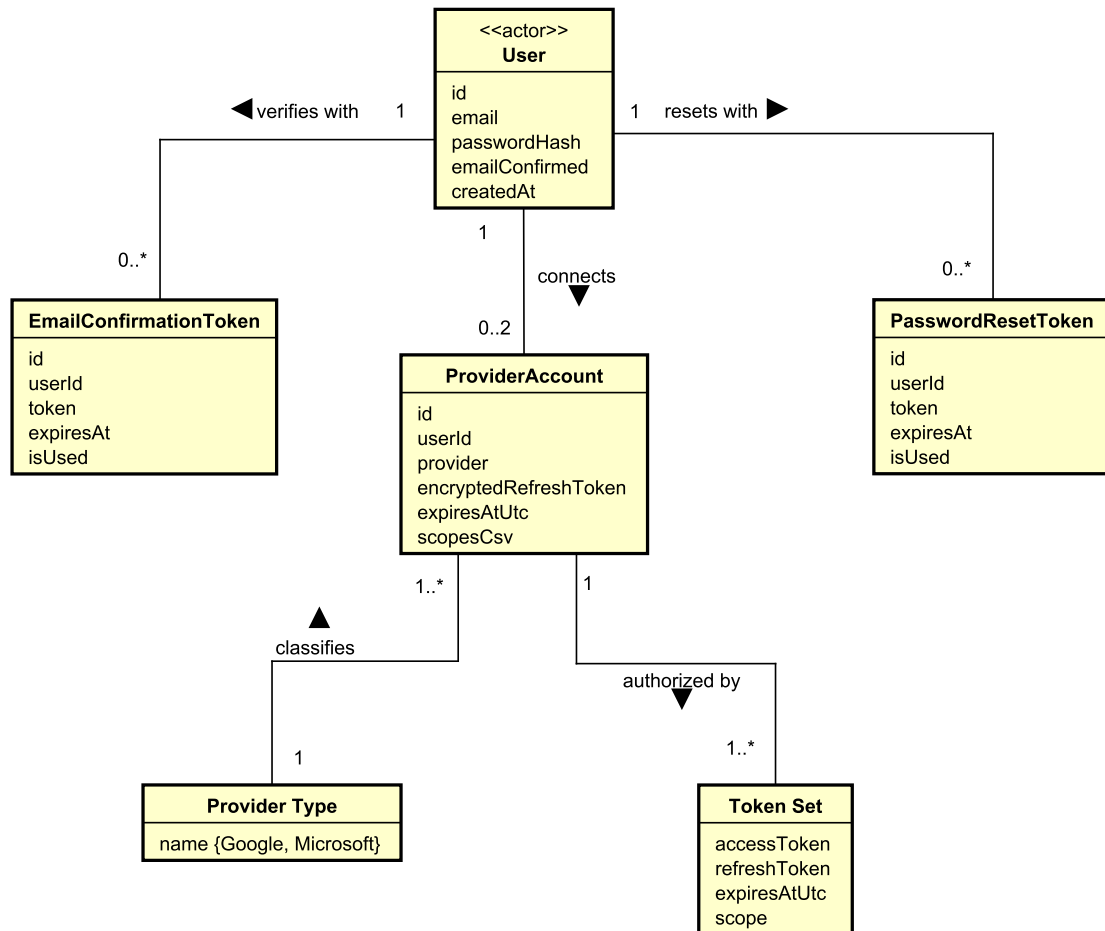
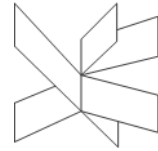


Figure 3: Domain model



3.6 Entity Relationship Diagram

The Entity Relationship Diagram reflects the domain model while incorporating a storage perspective. The *User* entity is the main table. The *EmailConfirmationToken* and the *PasswordResetToken* appear as time-limited and user-specific rows with primary and foreign keys. Each provider link is a *ProviderAccount* row which connects to both the user and a provider type, and stores just the encrypted refresh token, expiry and scopes. The *ProviderType* is kept as an enumeration. The *TokenSet* is represented as a temporary OAuth value which is not stored and only its refresh token, expiry and scopes are kept in the *ProviderAccount*.

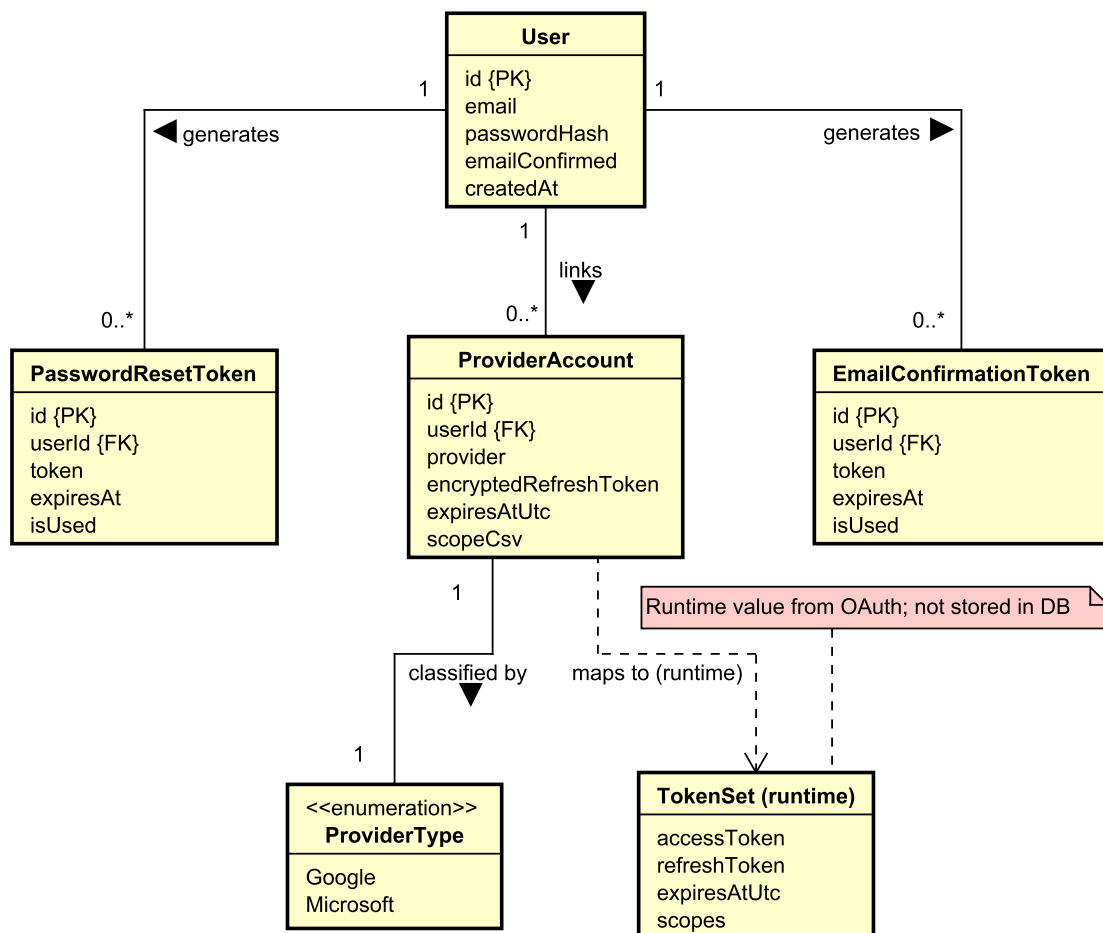
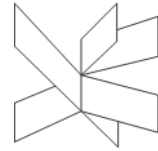


Figure 4: Entity relationship diagram



3.7 Security

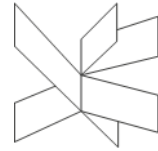
Security focuses on protecting credentials/tokens, enforcing per-user access, and containing incidents.

Key threats are:

- credential theft
- brute force on auth flows
- OAuth token theft/replay
- MITM (men-in-the-middle) on API/OAuth callbacks
- authorization bypass with stolen tokens
- DoS on gateway/services
- cross-tenant data access

The risk assessment identifies several threats: man-in-the-middle attacks (medium likelihood and high impact if plain HTTP is permitted), brute force attacks on authentication endpoints (medium likelihood and medium to high impact), token replay or theft (low to medium likelihood and high impact), denial-of-service attacks (medium likelihood and medium impact), and cross-tenant access (low likelihood and high impact). The planned controls include enforcing TLS exclusively, implementing rate limiting on sensitive endpoints, storing hashed passwords with time-limited single-use confirmation or reset tokens, utilizing short-lived JSON Web Tokens (JWTs), applying Proof Key for Code Exchange (PKCE) for OAuth, avoiding long-term access token storage, enforcing per-user authorization at service boundaries, isolating services and databases, adopting resilience patterns such as health checks and bounded retries or circuit breakers, and using correlation IDs in logging.

Threat	Likelihood	Impact	Planned Controls
Man-in-the-middle (if plain HTTP allowed)	Medium	High	Enforce TLS-only; security headers; CORS; correlation/logging



Brute force on auth endpoints	Medium	Medium/High	Rate limiting; strong password policy; time-limited tokens
Token replay/theft (JWT/OAuth)	Low/Medium	High	Short-lived JWTs; PKCE; no stored access tokens; introspection
Denial of Service (gateway/services)	Medium	Medium	Rate limits; bounded retries/circuit breakers; health checks
Cross-tenant data access	Low	High	Per-user authorization; service/database isolation; audit/logs

Table 3: Risk assessment

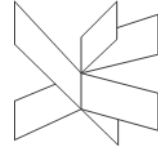
4. Design

4.1 Architectural overview

The system is split into two bounded services, *IdentityService* for authentication/user lifecycle and *LinkingService* for provider connections/unified file access, fronted by a single API Gateway and consumed by a web client. Each service owns its domain and data, runs independently, and can evolve or scale without affecting others. The gateway is the sole external entry point with a stable, versioned API and shared cross-cutting policies (auth validation, routing, resilience). This separation keeps cohesion high, coupling low, and contains failures within individual components.

4.2 Client

The client is a single-page web application that supports workflows including sign-up, sign-in, account confirmation, account recovery, provider linking, and file browsing. These workflows are organized into dedicated pages, and shared user interface components are used to maintain consistency. Navigation is handled exclusively on the



client side. All data interactions are handled through gateway-facing API calls, separating backend modifications from the user interface. Lightweight client-side state management maintains session context and tracks request outcomes. The user interface consistently communicates loading states, errors, and successful operations, ensuring clarity throughout authentication, provider linking, and file browsing workflows.

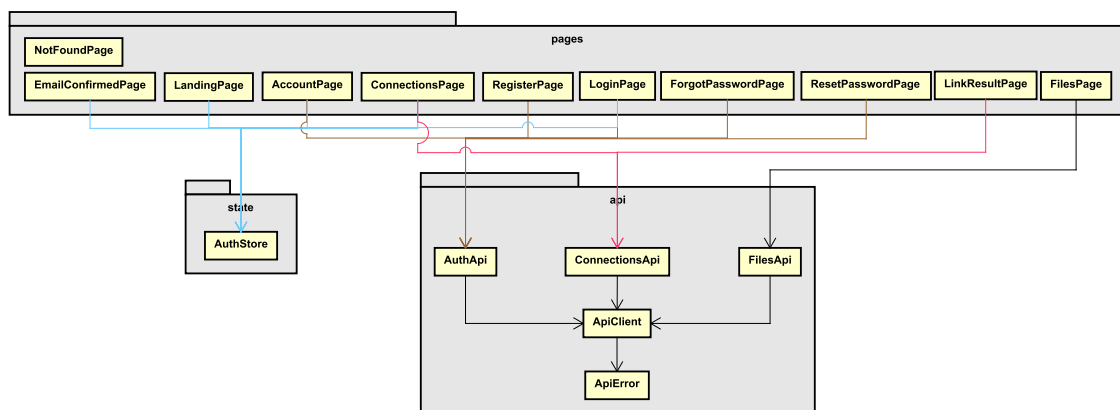


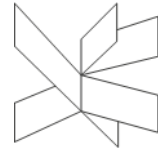
Figure 5: Simplified frontend class diagram

4.3 Gateway

The gateway is the single-entry point for all client traffic, providing a stable and versioned API while efficiently routing requests to backend services. At the network edge, it applies policies including authentication, Cross-Origin Resource Sharing (CORS), security headers, and logging, as well as resilience mechanisms such as timeouts, retries, and circuit breakers. Employing a single reverse proxy for clients hides internal services, simplifies connections, and centralizes the management of policy and routing modifications.

4.4 Services

Services follow their domain boundaries: *IdentityService* handles authentication and user lifecycle; *LinkingService* handles provider linking and unified file access. Each service owns its logic and persistence, exposes a REST interface, and validates authorization at its boundary. Independence allows separate scaling and evolution without impacting peers.



4.5 Persistence

Data is partitioned by service to align with domain ownership. The *IdentityService* stores user accounts along with their confirmation and reset tokens, while the *LinkingService* manages provider account links, including provider type, encrypted refresh tokens, expiry, and scopes. Utilizing separate databases enables independent schema modifications and scaling. Consistency across services is ensured through well-defined API contracts instead of shared schemas.

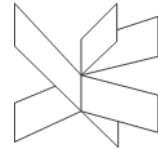
4.6 Security

Security controls described in the analysis are applied at the edge and within services: the gateway validates tokens and enforces origin/security headers; each service re-validates tokens per call, uses PKCE for OAuth, and stores only encrypted refresh tokens with scopes/expiry.

5. Implementation

5.1 Gateway edge (routing, authentication, resilience)

The API Gateway validates JWTs, applies CORS/security headers, adds resilience (retry/circuit breaker), exposes health endpoints, and hides internal service URLs. The *appsettings.json* declares CORS origins, reverse-proxy routes, and cluster destinations. The *Program.cs* reads that configuration, wires JWT/CORS and correlation, and maps health checks and the reverse proxy. In short, the *appsettings.json* defines what to forward and where, while the *Program.cs* enforces security/resilience and activates those routes.



```
// JWT Authentication
var jwtSecretKey = builder.Configuration["Jwt:SecretKey"] ??
    throw new InvalidOperationException("JWT SecretKey not configured");
var jwtIssuer = builder.Configuration["Jwt:Issuer"] ??
    "StorageConnector.IdentityService";
var jwtAudience = builder.Configuration["Jwt:Audience"] ??
    "StorageConnector";

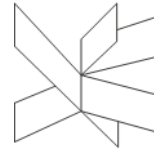
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSecretKey)),
            ValidateIssuer = true,
            ValidIssuer = jwtIssuer,
            ValidateAudience = true,
            ValidAudience = jwtAudience,
            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero
        };
    });

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("authenticated", policy => policy.RequireAuthenticatedUser());
});

// CORS
var allowedOrigins = builder.Configuration
    .GetSection("Cors:AllowedOrigins")
    .Get<string[]>() ?? ["https://localhost:5173"];

builder.Services.AddCors(options =>
{
    options.AddPolicy("Spa", policy =>
        policy.WithOrigins(allowedOrigins)
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials());
});
```

Figure 6: Gateway - JWT, CORS



```
// Resilience policies reused for proxy http clients
var circuitBreakerPolicy = HttpPolicyExtensions
    .HandleTransientHttpError()
    .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));

var retryPolicy = HttpPolicyExtensions
    .HandleTransientHttpError()
    .WaitAndRetryAsync(3, retryAttempt =>
        TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)));

var resiliencePolicy = Policy.WrapAsync<HttpResponseMessage>
    (retryPolicy, circuitBreakerPolicy);
// Health checks for the gateway itself
builder.Services.AddHealthChecks();
// YARP Reverse Proxy with resilience policies
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));
builder.Services.AddSingleton<IForwarderHttpClientFactory>(_ =>
    new ResilientForwarderHttpClientFactory(resiliencePolicy));
var app = builder.Build();

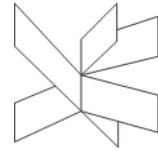
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
app.UseHttpsRedirection();
// Correlation ID middleware
app.Use(async (context, next) =>
{
    var correlationId = context.Request.Headers["X-Correlation-ID"].FirstOrDefault()
        ?? Guid.NewGuid().ToString();

    context.Items["CorrelationId"] = correlationId;
    context.Response.Headers["X-Correlation-ID"] = correlationId;

    await next();
});

app.UseCors("Spa");
app.UseAuthentication();
app.UseAuthorization();
// Gateway health check
app.MapHealthChecks("/health");
// Reverse proxy
app.MapReverseProxy();
app.Run();
```

Figure 7: Gateway - Resilience, YARP



5.2 LinkingService API

The linking API exposes two endpoints for the OAuth flow. The “*start*” endpoint is called by the client after the user clicks “connect”. It generates PKCE state and verifier, builds the provider’s authorize URL with the callback pointing back to this controller, and returns that URL for the client to redirect to. The “*callback*” endpoint is invoked by the provider after the user approves access. It validates the returned state, exchanges the code for tokens, stores the linked account (encrypted refresh token, expiry, scopes), and then redirects the user to the frontend success page (or an error page if validation fails).

```
[HttpGet("{provider}/start")]
0 references
public async Task<IActionResult> Start([FromRoute] ProviderType provider)
{
    var userId = User.RequireUserId();
    // Always include the API version segment so the callback hits this controller
    var redirect = new Uri($"{Request.Scheme}://{Request.Host}/api/v1/connect/{provider}/callback");
    var url = await _service.StartAsync(userId, provider, redirect, _scopes.For(provider));
    return Ok(new { redirectUrl = url.ToString() });
}
```

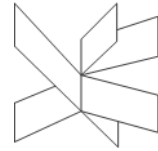
Figure 8: ConnectController - start

```
[HttpGet("{provider}/callback")]
[AllowAnonymous]
0 references
public async Task<IActionResult> Callback([FromRoute] ProviderType provider,
    [FromQuery] string state, [FromQuery] string code)
{
    // Must match the redirect URI registered with the provider
    var redirect = new Uri($"{Request.Scheme}://{Request.Host}/api/v1/connect/{provider}/callback");
    await _service.ConnectCallbackAsync(state, code, redirect);
    return Redirect($"{_frontendBaseUrl}/connections/success?provider={provider}");
}
```

Figure 9: ConnectController – callback

5.3 LinkingService core

This core flow covers state/PKCE, callback handling, and secure token persistence. *StartAsync* generates a PKCE verifier/challenge and random state, saves the tuple (*state*, *userId*, *provider*, *verifier*) in *CacheStateStore* with a short TTL, and returns the provider authorize URL. On callback, *ConnectCallbackAsync* consumes and validates the stored state/verifier, exchanges the code, and hands the resulting tokens to



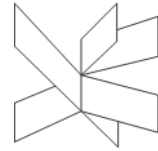
EfTokenStore to *upsert* the user's provider account. *EfTokenStore* encrypts the refresh token, stores expiry and scopes, and ensures one record per user/provider. Together, the state store prevents replay, and the token store keeps provider credentials safe at rest.

```
public async Task<Uri> StartAsync(Guid userId, ProviderType provider, Uri redirectUri, string[] scopes)
{
    var state = Base64Url(24);
    var verifier = Base64Url(32);
    var challenge = CodeChallenge(verifier);
    await _states.SaveAsync(state, userId, verifier, provider, TimeSpan.FromMinutes(10));
    return new Uri(C(provider).BuildAuthorizeUrl(state, challenge, redirectUri, scopes));
}
3 references
public async Task ConnectCallbackAsync(string state, string code, Uri redirectUri)
{
    var result = await _states.TakeAsync(state);
    if (result is null)
        throw new InvalidOperationException("State expired");
    var (userId, codeVerifier, provider) = result.Value;
    var tokens = await C(provider).ExchangeCodeAsync(code, codeVerifier, redirectUri);
    var acc = await _tokens.GetAsync(userId, provider) ??
        new ProviderAccount { UserId = userId, Provider = provider };
    acc.UpdateFrom(tokens, _tokens.Encrypt);
    await _tokens.UpsertAsync(acc);
}
```

Figure 10: LinkProviderService

```
4 references | 4 references
private readonly IMemoryCache _cache; public CacheStateStore(IMemoryCache c) => _cache = c;
5 references
public Task SaveAsync(string state, Guid userId, string cv, ProviderType p, TimeSpan ttl)
{ _cache.Set(state, (userId, cv, p), ttl); return Task.CompletedTask; }
5 references
public Task<(Guid userId, string codeVerifier, ProviderType provider)> TakeAsync(string state)
{
    if (_cache.TryGetValue<(Guid, string, ProviderType)>(state, out var v))
    { _cache.Remove(state); return Task.FromResult<(Guid, string, ProviderType)>(v); }
    return Task.FromResult<(Guid, string, ProviderType)>(null);
}
```

Figure 11: CacheStateStore



```

public sealed class EfTokenStore : ITokenStore
{
    8 references | 3 references
    private readonly DbContext _db; private readonly IDataProtector _p;
    2 references
    public EfTokenStore(DbContext db, IDataProtectionProvider dp) { _db = db; _p =
        dp.CreateProtector("tokens.v1"); }

    5 references
    public Task<ProviderAccount?> GetAsync(Guid userId, ProviderType provider) =>
        _db.Set<ProviderAccount>().FirstOrDefaultAsync(x => x.UserId == userId &&
            x.Provider == provider);

    2 references
    public async Task<IReadOnlyList<ProviderAccount>> GetAllByUserAsync(Guid userId) =>
        await _db.Set<ProviderAccount>().Where(x => x.UserId == userId).ToListAsync();

    4 references
    public async Task UpsertAsync(ProviderAccount account)
    {
        var set = _db.Set<ProviderAccount>();

        // Try to find an existing account for the same user+provider.
        var existing = await set.FirstOrDefaultAsync(x => x.UserId == account.UserId &&
            x.Provider == account.Provider);
        if (existing is null)
        {
            // New account -> add
            await set.AddAsync(account);
        }
        else
        {
            // Existing account -> copy values from the provided instance onto the tracked entity.
            _db.Entry(existing).CurrentValues.SetValues(account);
        }

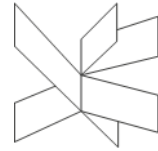
        await _db.SaveChangesAsync();
    }

    2 references
    public async Task DeleteAsync(Guid userId, ProviderType provider)
    {
        var a = await GetAsync(userId, provider); if (a != null)
        { _db.Remove(a); await _db.SaveChangesAsync(); }
    }

    5 references
    public string Encrypt(string plaintext) => _p.Protect(plaintext);
    7 references
    public string Decrypt(string ciphertext) => _p.Unprotect(ciphertext);
}

```

Figure 12: EfTokenStore



5.4 OAuth clients and refresh

MicrosoftOAuthClient handles the PKCE code exchange and refreshes tokens as needed, while *MicrosoftFileProvider* consumes those tokens to serve file operations. The file provider decrypts the stored refresh token, refreshes the access token when required, updates the saved refresh token/expiry/scopes, and then returns the provider's view URL for the requested file. Together they cover the token lifecycle: exchange, refresh, use, without storing access tokens at rest.

```
public async Task<TokenSet> ExchangeCodeAsync(string code, string codeVerifier, Uri redirectUri)
{
    var payload = new Dictionary<string, string>
    {
        ["client_id"] = _options.ClientId,
        ["client_secret"] = _options.ClientSecret,
        ["grant_type"] = "authorization_code",
        ["code"] = code,
        ["redirect_uri"] = redirectUri.ToString(),
        ["code_verifier"] = codeVerifier
    };

    var response = await _http.PostAsync(_tokenEndpoint, new FormUrlEncodedContent(payload));
    var json = await ReadJsonAsync<MicrosoftTokenResponse>(response);

    if (string.IsNullOrEmpty(json.RefreshToken))
    {
        throw new InvalidOperationException(
            "Microsoft did not return a refresh token. Ensure offline_access scope is granted.");
    }

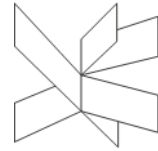
    return ToTokenSet(json, json.RefreshToken);
}

6 references
public async Task<TokenSet> RefreshAsync(string refreshToken)
{
    var payload = new Dictionary<string, string>
    {
        ["client_id"] = _options.ClientId,
        ["client_secret"] = _options.ClientSecret,
        ["grant_type"] = "refresh_token",
        ["refresh_token"] = refreshToken
    };

    var response = await _http.PostAsync(_tokenEndpoint, new FormUrlEncodedContent(payload));
    var json = await ReadJsonAsync<MicrosoftTokenResponse>(response);

    return ToTokenSet(json, json.RefreshToken ?? refreshToken);
}
```

Figure 13: *MicrosoftOAuthClient*



```
private async Task<string> GetAccessTokenAsync(string userId)
{
    if (!Guid.TryParse(userId, out var userGuid))
    {
        throw new InvalidOperationException($"Invalid user ID format: {userId}");
    }

    var account = await _tokenStore.GetAsync(userGuid, ProviderType.Microsoft);
    if (account is null)
    {
        throw new InvalidOperationException($"No Microsoft account linked for user {userId}");
    }

    var oauthClient = _oauthClients.FirstOrDefault(c => c.Provider == ProviderType.Microsoft)
        ?? throw new InvalidOperationException("Microsoft OAuth client not registered");

    // Check if token is expired (with 5 min buffer)
    if (account.ExpiresAtUtc <= DateTimeOffset.UtcNow.AddMinutes(5))
    {
        _logger.LogInformation("Access token expired, refreshing for user {UserId}", userId);

        // Decrypt refresh token and get new access token
        var refreshToken = _tokenStore.Decrypt(account.EncryptedRefreshToken);
        var refreshed = await oauthClient.RefreshAsync(refreshToken);

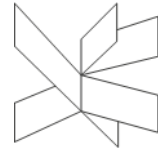
        // Update the stored account with new token data
        account.UpdateFrom(refreshed, _tokenStore.Encrypt);
        await _tokenStore.UpsertAsync(account);

        return refreshed.AccessToken;
    }

    // Token not expired - still need to refresh to get access token since we don't store it
    var decryptedRefreshToken = _tokenStore.Decrypt(account.EncryptedRefreshToken);
    var tokenResponse = await oauthClient.RefreshAsync(decryptedRefreshToken);

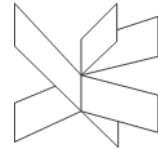
    return tokenResponse.AccessToken;
}
```

Figure 14: MicrosoftFileProvider



5.5 File aggregation

FilesController exposes a unified file surface: *GetAllFiles* pulls every linked provider for the current user and returns a merged list; *GetFilesByProvider* supports per-provider listing with folder and paging; *GetFileMetadata* fetches details for a single item; and the view endpoints return either a redirect or a JSON view URL. All these delegate to *FileService*. In *FileService*, *GetFilesFromAllProvidersAsync* enumerates the user's linked accounts, asks the appropriate file provider for each list, flattens and sorts the results by last modified time, and tolerates per-provider errors without breaking the whole aggregation. The per-provider methods *GetFilesByProviderAsync*, *GetFileMetadataAsync*, *GetFileViewUrlAsync* simply forward to the specific provider via the factory, handling pagination, metadata retrieval, and generation of the provider's view URL.



```
// Get files from all linked providers (unified dashboard view).
[HttpGet]
0 references
public async Task<ActionResult<IReadOnlyList<ProviderFileItem>>> GetAllFiles(
    [FromQuery] int pageSize = 50)
{
    var userId = User.RequireUserId();

    var files = await _fileService.GetFilesFromAllProvidersAsync(userId, pageSize);

    _logger.LogInformation("Returned {Count} files for user {UserId}", files.Count, userId);

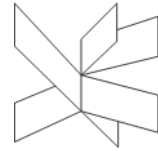
    return Ok(files);
}

// Get files from a specific provider with pagination support.
[HttpGet("{provider}")]
0 references
public async Task<ActionResult<FileListResponse>> GetFilesByProvider(
    ProviderType provider,
    [FromQuery] string? folderId = null,
    [FromQuery] int pageSize = 50,
    [FromQuery] string? pageToken = null)
{
    var userId = User.RequireUserId();

    var (items, nextPageToken) = await _fileService.GetFilesByProviderAsync(
        userId, provider, folderId, pageSize, pageToken);

    return Ok(new FileListResponse(items, nextPageToken));
}
```

Figure 15: FilesController - GetAllFiles, GetFilesByProvider



```
// Get detailed metadata for a specific file.
[HttpGet("{provider}/{fileId}/metadata")]
0 references
public async Task<ActionResult<FileMetadata>> GetFileMetadata(
    ProviderType provider,
    string fileId)
{
    var userId = User.RequireUserId();

    var metadata = await _fileService.GetFileMetadataAsync(userId, provider, fileId);

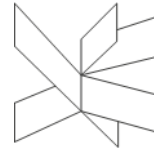
    return Ok(metadata);
}

// Get the web view URL for a file (redirects to provider's web interface).
[HttpGet("{provider}/{fileId}/view")]
0 references
public async Task<ActionResult> GetFileViewUrl(
    ProviderType provider,
    string fileId)
{
    var userId = User.RequireUserId();

    var viewUrl = await _fileService.GetFileViewUrlAsync(userId, provider, fileId);

    return Redirect(viewUrl.ToString());
}
```

Figure 16: FilesController - GetFileMetadata, GetFileViewUrl



```

public sealed class FileService : IFileService
{
    public async Task<IReadOnlyList<ProviderFileItem>> GetFilesFromAllProvidersAsync(
        Guid userId, int pageSize = 50)
    {
        // Get all linked accounts for the user
        var linkedAccounts = await _tokenStore.GetAllByUserAsync(userId);

        if (!linkedAccounts.Any())
        {
            _logger.LogInformation("User {UserId} has no linked accounts", userId);
            return Array.Empty<ProviderFileItem>();
        }

        // Fetch files from each provider in parallel
        var tasks = linkedAccounts.Select(async account => ...

        var results = await Task.WhenAll(tasks);

        // Flatten and sort by modified date (newest first)
        var allFiles = results
            .SelectMany(files => files)
            .OrderByDescending(f => f.ModifiedUtc)
            .ToList();

        _logger.LogInformation(
            "Aggregated {Count} files from {ProviderCount} providers for user {UserId}",
            allFiles.Count, linkedAccounts.Count, userId);

        return allFiles;
    }

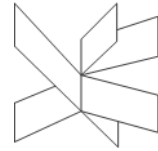
    2 references
    public async Task<IReadOnlyList<FileItem>> GetFilesByProviderAsync(
        Guid userId, ProviderType provider, string? folderId = null,
        int pageSize = 50, string? pageToken = null)
    {
        var fileProvider = _fileProviderFactory.Get(provider);
        return await fileProvider.ListAsync(userId.ToString(), folderId, pageSize, pageToken);
    }

    2 references
    public async Task<FileMetadata> GetFileMetadataAsync(Guid userId, ProviderType provider, string fileId)
    {
        var fileProvider = _fileProviderFactory.Get(provider);
        return await fileProvider.GetMetadataAsync(userId.ToString(), fileId);
    }

    3 references
    public async Task<Uri> GetFileViewUrlAsync(Guid userId, ProviderType provider, string fileId) ...
}

```

Figure 17 :FileService



5.6 Identity touchpoints

AuthController issues and verifies the tokens that guard the linking flow. The register/login endpoints create users, enforce email confirmation, validate credentials, and issue JWTs with the expected issuer/audience/claims. The introspection endpoint allows other services to validate a presented token: it checks signature, issuer/audience/lifetime, extracts the subject/email, and confirms the user still exists. These touchpoints define the token shape and validation contract that the gateway and *LinkingService* rely on.

5.7 Resilience and security

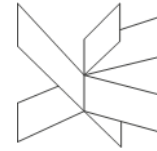
Both services enforce security and resilience at the edge. In their *Program* file, they configure JWT validation (issuer/audience/secret, zero clock skew), apply IP rate limiting to sensitive endpoints, and add middleware for correlation IDs and security headers. They also expose health checks and wire CORS for the SPA. These settings ensure tokens are revalidated on every call, request origins are controlled, and basic hardening and observability (correlation/logging) are in place alongside controlled retries/limits to reduce abuse.

6. Testing

6.1 Acceptance testing

Each user case was validated through acceptance testing. For the use case “Connect provider’s account,” acceptance testing covered the full OAuth flow for both Google and Microsoft with a logged-in user. In the base case, the user starts the link, approves consent, and the callback returns a valid code. The system exchanges the code, stores tokens encrypted and shows the connection as “Connected.” The edge cases were also covered:

- user denies consent (returned to the app with status “Not connected”)
- callback contains an empty/invalid code (no tokens stored, “Reconnect” prompt shown)



- provider returns an error during token exchange (error shown, status remains “Not connected”).

In each case the observed outcome matched the expected state and messaging.

Scenarios to cover					
Base:	User consents	Code received	Token stored (access + refresh)	State = Connected	
ALT1	Denies consent				
ALT2	Code missing/invalid on callback				
ALT3	Token exchange/storage fails (network, provider error)				
TC1 - Consent OK					
Step 1:	Start connect	Consent = Allow	Callback Code = Valid	Expected result: Tokens saved (encrypted), status Connected	Actualt result: Same
TC2 - Deny					
Step 1:	Start connect	Consent = Deny		Expected result: Back to app with "Not connected"	Actualt result: Same
TC3 - Invalid code					
Step 1:	Start connect	Consent = Allow	Code = empty/invalid	Expected result: "Reconnect" prompt, no token stored	Actualt result: Same
TC4 - Exchange fails					
Step 1:	Start connect	Consent = Allow	Provider 500 during token exchange	Expected result: Error shown, status Not connected	Actualt result: Same

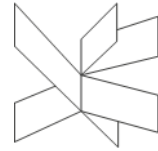
Figure 18: Acceptance testing - connect provider's account

6.2 Backend testing

IdentityService is covered by *xUnit* unit and integration tests for registration/login, email confirmation, password reset, token issuance/introspection, and rate-limit/error handling. *LinkingService* has unit, integration, and E2E tests for PKCE/state handling, OAuth callback validation, token storage/encryption, connection status/disconnect, file aggregation and view URL generation, and resilience/error handling when a provider fails.

6.3 Frontend testing

Unit/component tests with *Vitest* and *React Testing Library* cover the register/login/reset/forgot flows, connections page link/disconnect states, and files page loading/error/empty states and search/filter. Lightweight integration checks mock the API clients to verify success/error rendering, token handling, and navigation guards.



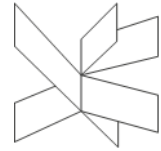
6.4 Summary of test coverage

All automated backend suites pass, and the frontend suites do as well. Manual acceptance tests across the main user journeys also completed as expected, with proper handling of invalid tokens, denied consent, and provider/API failures.

7. Discussion and conclusion

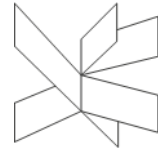
The solution met its core aims: a gateway-fronted, two-service architecture that keeps the client insulated from internal topology, with clear ownership of authentication and provider linking/file aggregation. The linking flow proved robust—PKCE/state guarded against replay, access tokens remained transient, and file aggregation provided a unified view across providers. Tests covered *IdentityService*'s token lifecycle and *LinkingService*'s OAuth/state handling, plus frontend flows for auth, linking, and files. Acceptance testing matched expected outcomes in both success and error paths.

Gaps remain: the system is read-only and the UI doesn't yet surface folder navigation or richer metadata. Transient access tokens require refresh on each operation. development choices (in-memory state cache and databases) need production hardening. End-to-end load and security testing are still pending, and rate limiting must be tuned for production. CI/CD was not included in the project, but adding a pipeline would make sense for automated build/test and deployment once the application is ready. Next steps include extending the UX (navigation, file management), adding providers beyond Google/Microsoft, hardening infrastructure (persistent state store, production databases, secret vaulting, observability), and adding load/security testing and CI/CD to validate and deploy at scale.



8. References

- Barry, P. (24.12.2023). Retrieved March 21 from Adivi [Why is Cloud Storage Important for Businesses?](#)
- Gartner. Retrieved March 21 from Gartner [Multicloud Strategy](#)

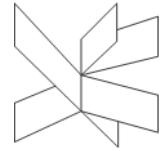


List of Figures

Figure 1: Use case diagram	5
Figure 2: System sequence diagram - Link provider's account.....	8
Figure 3: Domain model.....	9
Figure 4: Entity relationship diagram.....	10
Figure 5: Simplified frontend class diagram	13
Figure 6: Gateway - JWT, CORS.....	15
Figure 7: Gateway - Resilience, YARP	16
Figure 8: ConnectController - start.....	17
Figure 9: ConnectController – callback	17
Figure 10: LinkProviderService	18
Figure 11: CacheStateStore.....	18
Figure 12: EfTokenStore.....	19
Figure 13: MicrosoftOAuthClient	20
Figure 14: MicrosoftFileProvider	21
Figure 15: FilesController - GetAllFiels, GetFilesByProvider	23
Figure 16: FilesController - GetFileMetadata, GetFileViewUrl	24
Figure 17 :FileService	25
Figure 18: Acceptance testing - connect provider's account.....	27

List of Tables

Table 1: Requirements coverage	6
Table 2: Use case description - Link provider's account.....	7
Table 3: Risk assessment.....	12



Appendices

1. Project description
2. Acceptance testing
3. Source code
4. Diagrams
5. Use case descriptions
6. Requirements
7. GitHub link
8. Video Link
9. Product backlog
10. Interpersonal Skills
11. Individual Project Contract